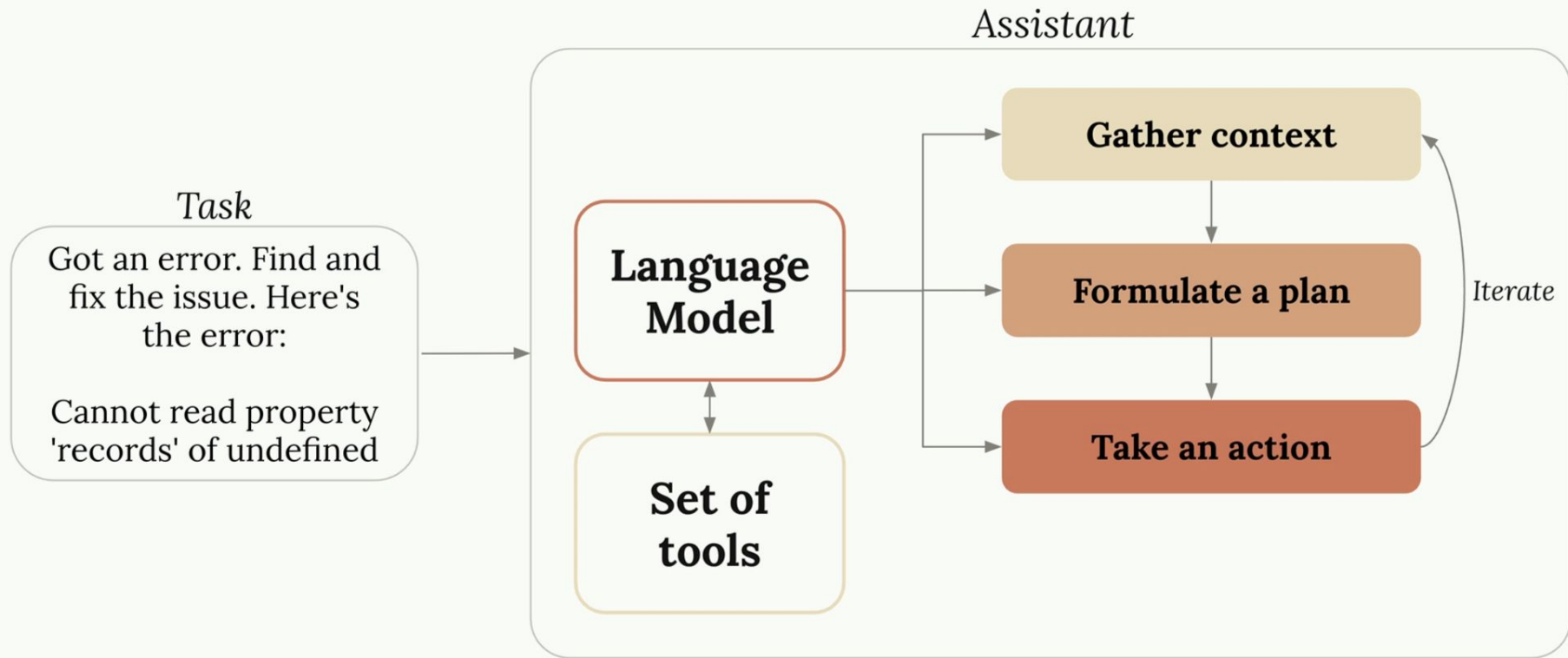


Claude Code in Action

What is Claude Code?

How do Coding Assistants work?

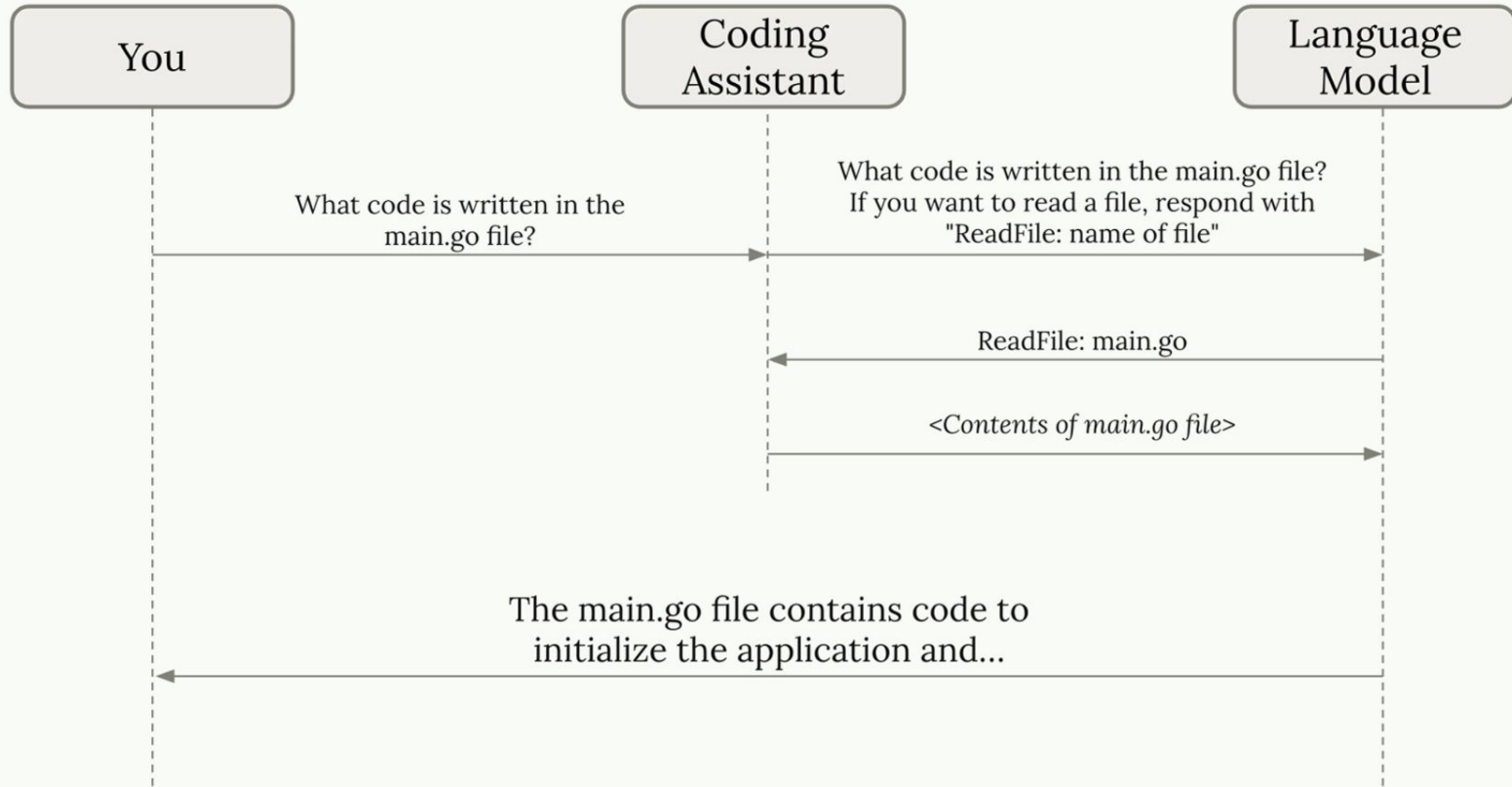


- Most coding assistants use remotely hosted language models
- Claude Code uses the Claude series of models hosted at Anthropic, AWS, or Google Cloud (configurable)

How do Coding Assistants work?

- Coding assistant passes off the task to debug an error internally to language model
- Language model comes up with an approach to solve the task like a human would:
 - Gather Context: understand what the error is referring to, area of the codebase throwing the error, and what code files seem to be relevant
 - Formulate Plan: how to accomplish the task—change the code and then run or write a test to verify issue is actually fixed
 - Take Action: updating a file and running a test
- Gathering context and taking actions require coding assistants to interact with the outside world—read/edit file, fetch online documentation, run a command, etc.

Tool Use



Tool Use

- Coding assistant appends system message to the user request and calls LLM
- LLM understands the action to perform to serve user request and generate message for the action (tool-use) in expected format
- Coding assistant receives formatted message and understands LLM wants to take an action by reading a file
- Coding assistant reads the file contents and passes it back to the LLM
- LLM reads the contents of the file and generates the final response for user query
- System of giving LLM extra instructions, asking it to respond in a carefully formatted way is referred to as **Tool Use**

Tool Use

- Tools are used to give LLMs extra capabilities to interact with outside world
- In summary:
 - Models are given plain text instructions on how to respond a certain way to use a tool
 - When the model responds with a request to use a tool, coding assistant does whatever the tool is supposed to do (read / write files, make a request, etc.)
- Claude model series—Sonnet, Opus, Haiku are particularly strong at understanding what tools do and using them to effectively complete tasks
- These models can also use tools in interesting combinations to accomplish more advanced & complex tasks

Core Strength: Strong Tool Use

Claude Code can tackle harder tasks

- Claude will eagerly combine together different tools to handle more complex work
- Claude will adeptly use tools it hasn't seen before

Claude Code is extensible

- You can easily add additional tools (capabilities) to Claude Code
- Adding tools allows customization for your particular workflow
- Adding tools allow Claude Code to keep up with rapid changes in AI-enabled development

Improved security

- Thanks to Claude's strong tool use, Claude Code has can easily navigate codebases
- Claude Code doesn't rely upon indexing your codebase, which often requires sending your entire codebase to outside servers

Summary

- Coding assistants use language models to complete different tasks
- Language models need to know how to use tools on vast majority of tasks. These tools are used to read files, write files, run commands, and everything else outside of just generating text
- Not all language models use tools with the same finesse and this has a big impact on the overall efficiency of the coding assistant
- Claude's strong tool use with Claude Code allows for better security, customization, and longevity

Claude Code Tasks

Claude Code: Default Tools

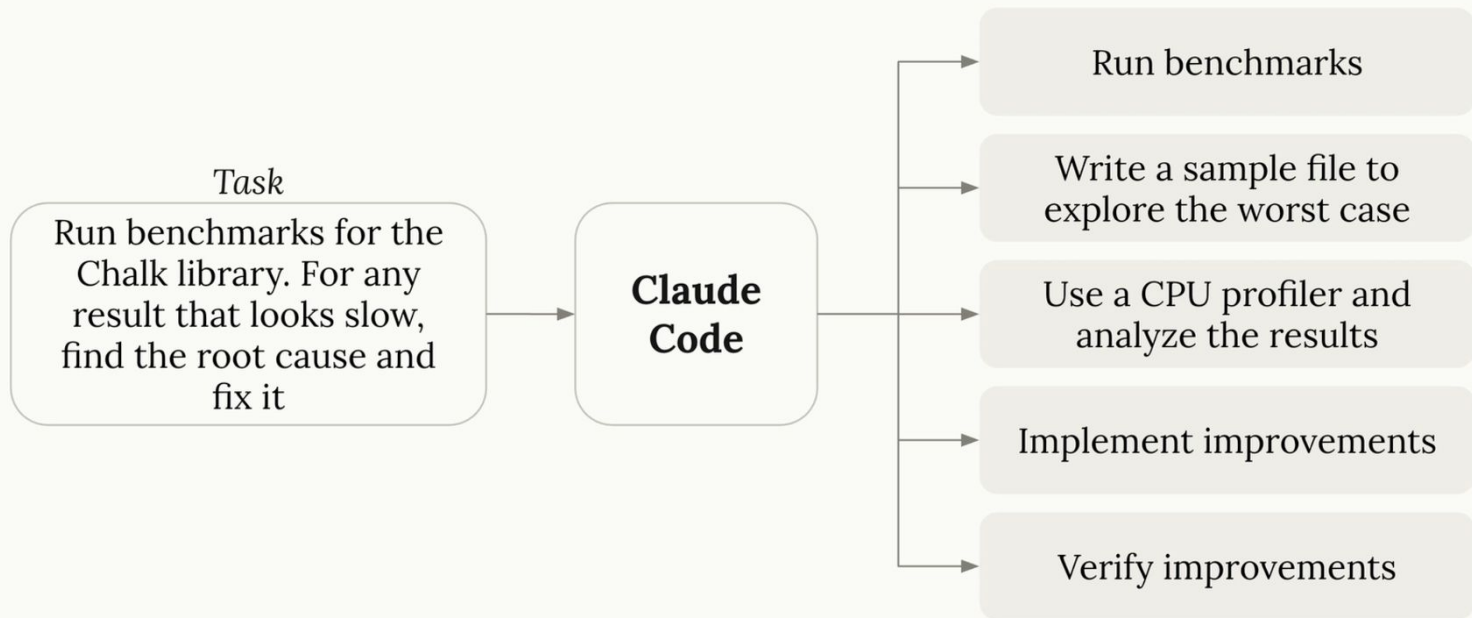
Name	Purpose
Agent	Launch a subagent to handle a task
Bash	Run a shell command
Edit	Edit a file
Glob	Find files based on a pattern
Grep	Search the contents of a file
LS	List files and directories
MultiEdit	Make several edits at the same time
NotebookEdit	Write a cell in a Jupyter notebook
NotebookRead	Read a cell

Name	Purpose
Read	Read a file
ToDoRead	Read one of the created to-do's
ToDoWrite	Update the list of to-do's
WebFetch	Fetch from a URL
WebSearch	Search the web
Write	Write to a file

Claude Code: Tasks

Optimization Task

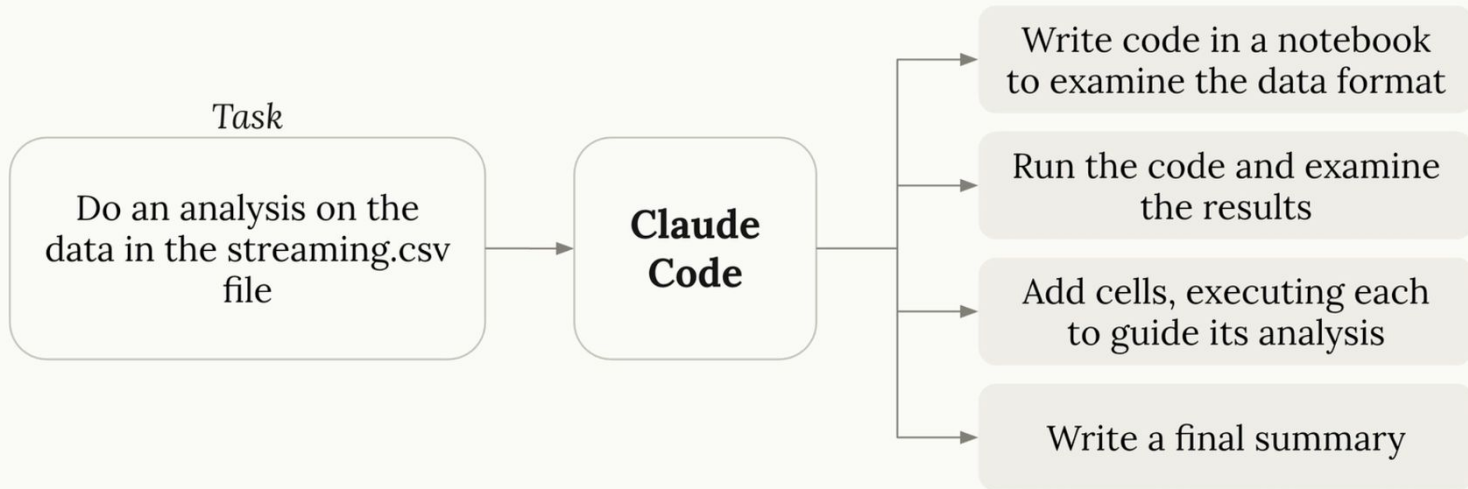
Find performance issues and optimize the Chalk library



Claude Code: Tasks

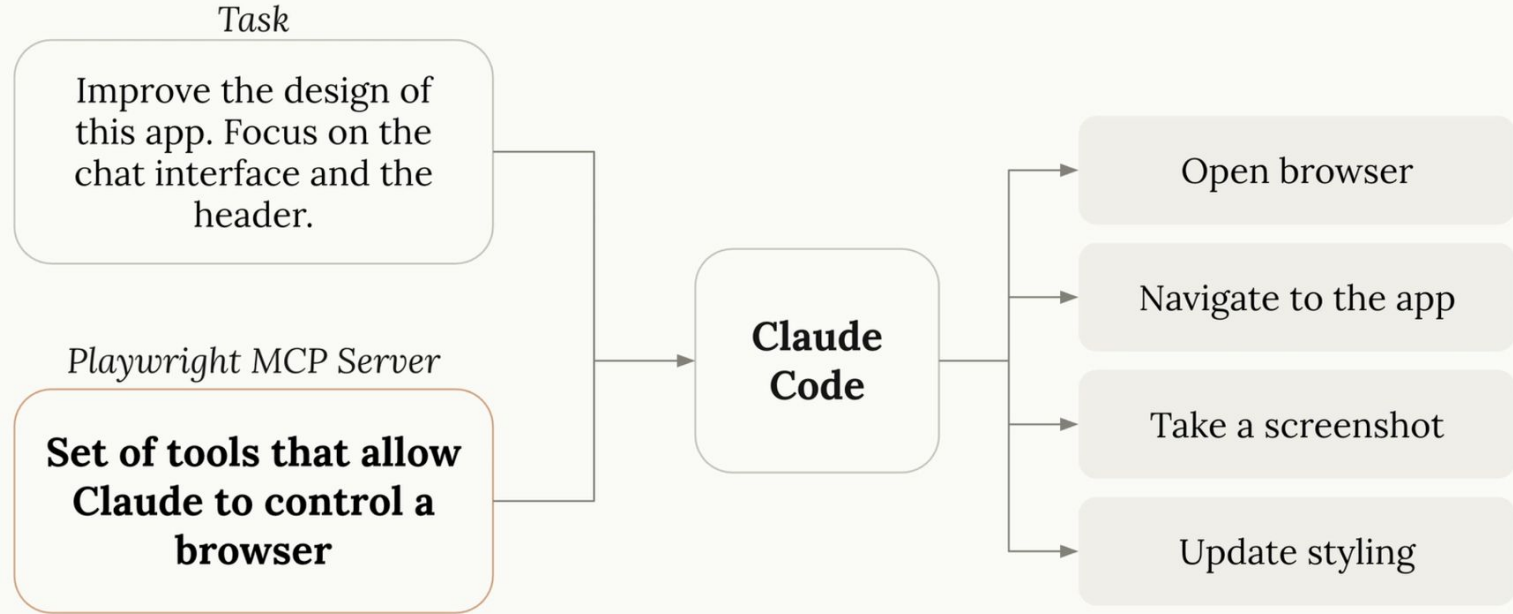
Data analysis task

Find insights using data about video streaming platform users

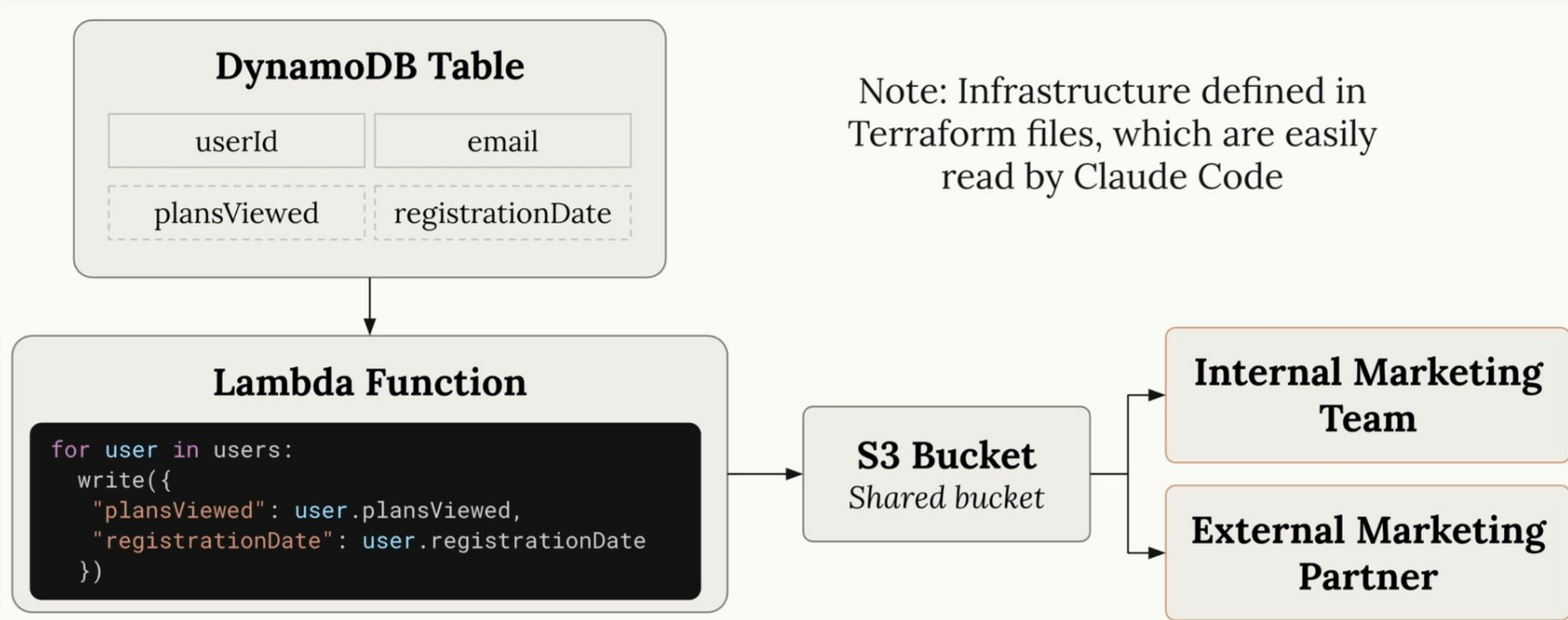


Claude Code: Tasks

UI styling task

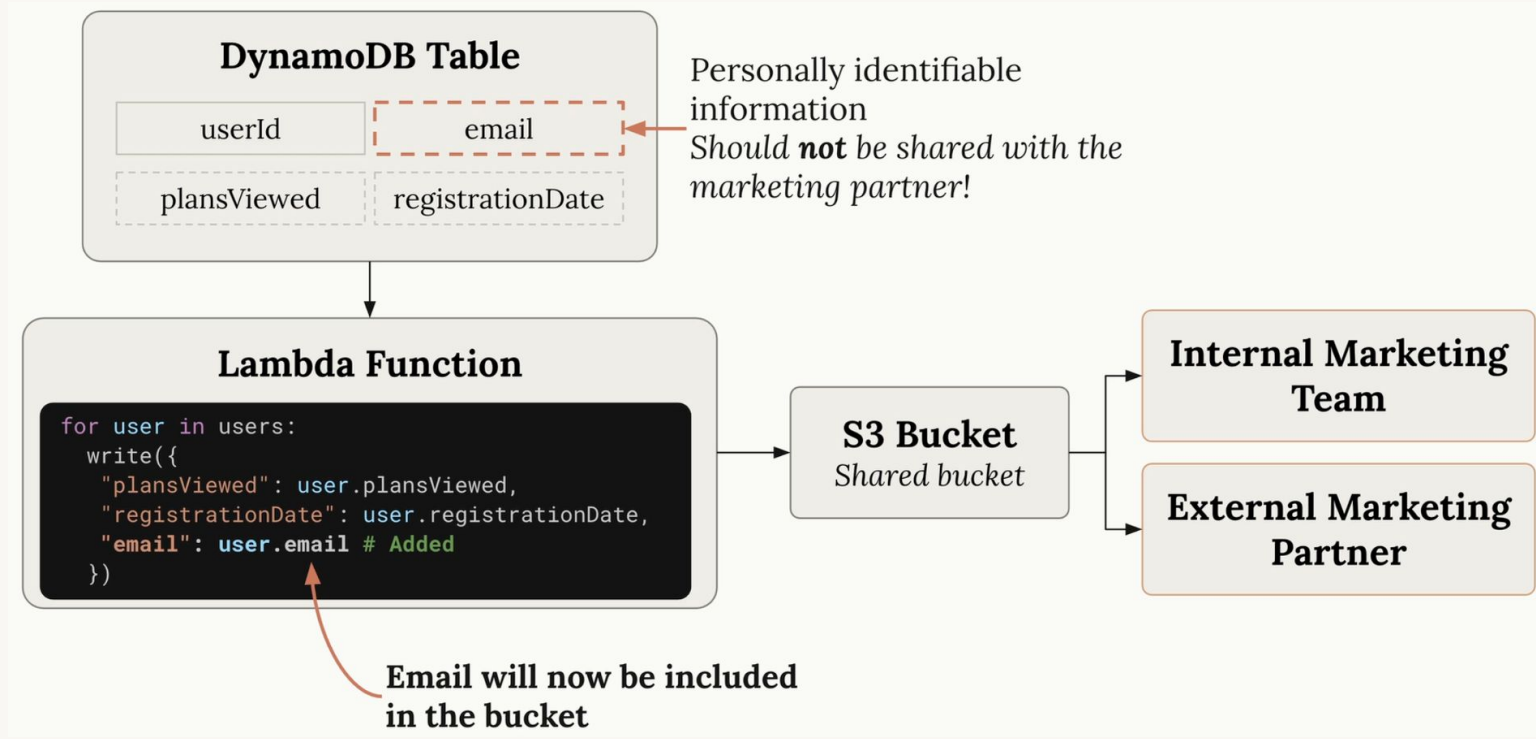


Claude Code: Tasks



- Claude has a good idea of information flow from infrastructure defined in Terraform files
- Lambda function pulls user data from DynamoDB and writes to S3 which is shared with internal and external marketing teams

Claude Code: Tasks



- Months later, internal marketing team requests to also add email field to the S3 bucket
- S3 data is also shared with external marketing team and may get overlooked
- PII exposed to external company. This occurs at times and is hard to catch
- Claude Code can catch these kind of scenarios inside a Pull Request based on the terraform files

Claude Code: Tasks

 **Feat: User Processing #10**
StephenGrider wants to merge 2 commits into `main` from `users` 

 **claude** bot commented 3 days ago · edited ...

Claude finished [@StephenGrider's](#) task — [View job](#)

Security Review of PR #10: User Processing

Critical security issue identified with PII exposure to partner accounts.

Tasks:

- ☒ Use git commands to understand exact changes made
- ☒ Read the modified lambda function to understand email processing
- ☒ Review workflow file changes
- ☒ Analyze terraform infrastructure configuration for PII handling
- ☒ Check for partner account access and PII exposure risks
- ☒ Provide comprehensive security review feedback

 **CRITICAL SECURITY ISSUE: PII EXPOSURE TO PARTNER ACCOUNTS**

Issue: This PR introduces a serious security vulnerability by adding user email addresses (PII) to data that is automatically shared with external partner accounts.

Detailed Analysis:

 **CRITICAL SECURITY ISSUE: PII EXPOSURE TO PARTNER ACCOUNTS**

Issue: This PR introduces a serious security vulnerability by adding user email addresses (PII) to data that is automatically shared with external partner accounts.

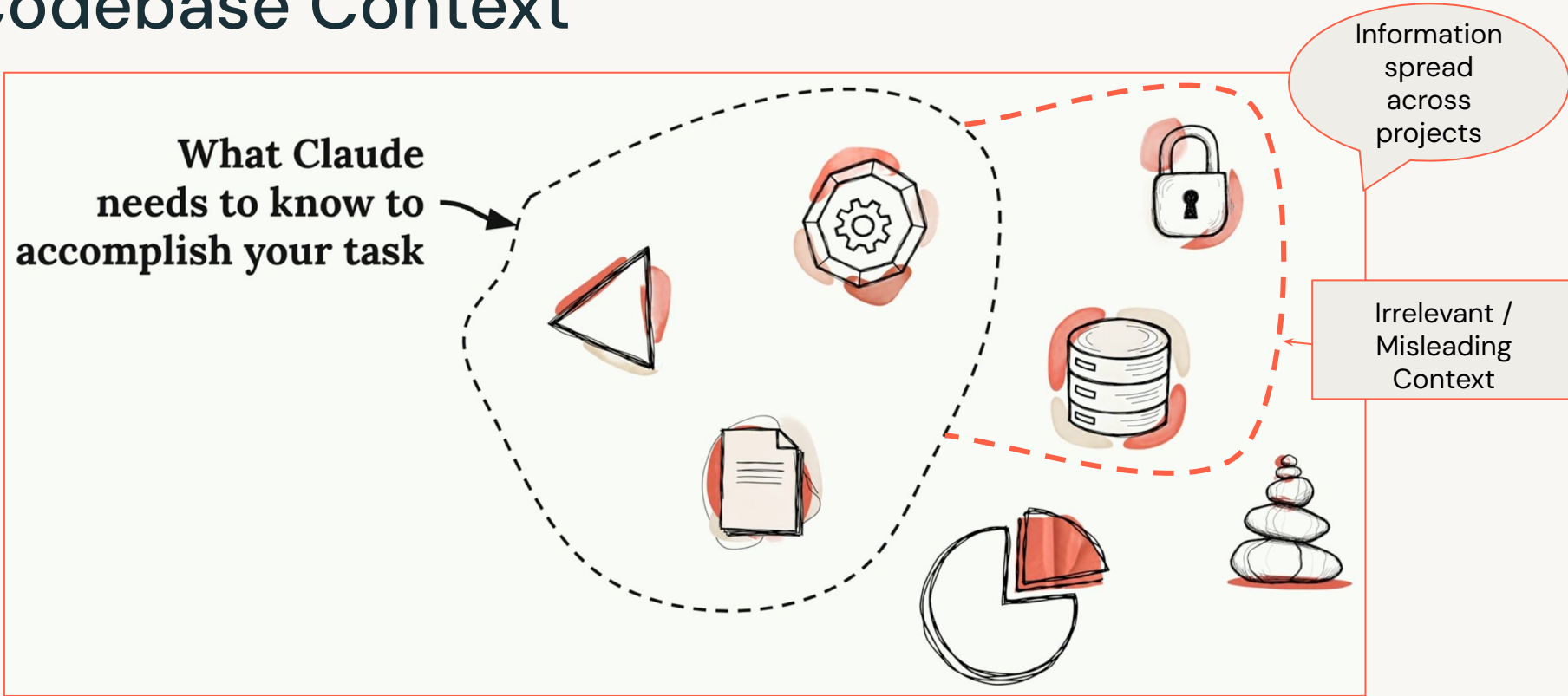
Detailed Analysis:

- PII Data Flow:**
 - The lambda function (`infrastructure/data-processing/lambda/lambda_function.py:95`) now includes user email addresses in processed data
 - This data is stored in the analytics S3 bucket (`data.terraform_remote_state.analytics.outputs.analytics_bucket_id`)
 - The analytics bucket has cross-account access configured for partner account `927374824227`
- Partner Account Access:**
 - The analytics S3 bucket policy (`infrastructure/analytics/modules/s3-analytics-bucket/main.tf:114-133`) grants the partner account:
 - `s3:GetObject` - Can read all files including PII data
 - `s3:ListBucket` - Can list all objects in the bucket
 - `s3:GetBucketLocation` and `s3:GetBucketVersioning` - Metadata access
 - Partner account has ROOT access (`arn:aws:iam::927374824227:root`) with minimal restrictions
- Data Storage Location:**
 - PII data is stored in path: `user-data/{environment}/year={year}/month={month}/day={day}/user_data_{timestamp}.json`

- Claude Code reviews the pull request and identifies PII exposure to an external partner
- Lists dataflow in detail and how this bucket is shared with the partner
- Issues can be flagged during development rather than post deployment

Getting Hands-On

Codebase Context



- Inside a project, there maybe hundreds of files with tremendous amount of information
- To accomplish a specific task, there's an ideal amount of information that Claude needs
- As we add more information that's not relevant for the task, Claude Code's effectiveness decreases
- Guide Claude towards relevant files inside the project with some guidance to get best results

/init

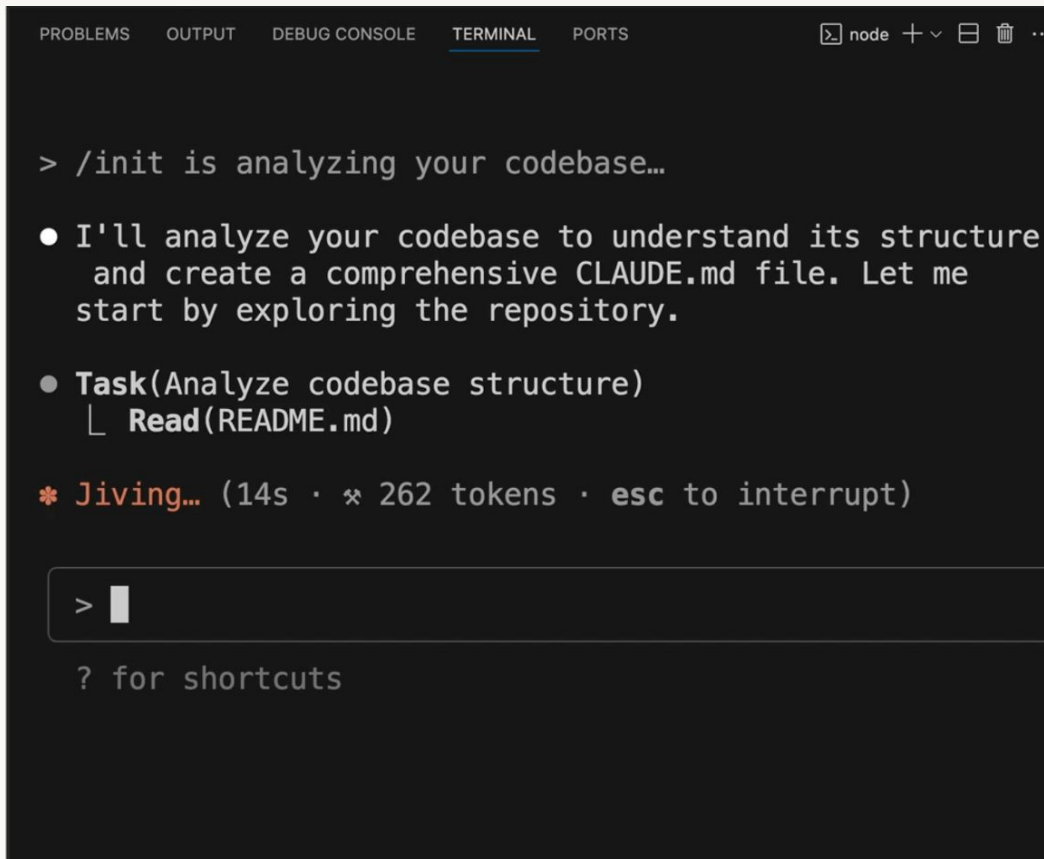
The /init command

Scans your codebase

Creates a summary

Writes the summary to the
CLAUDE.md file

This file is included in every
request



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
> /init is analyzing your codebase...

• I'll analyze your codebase to understand its structure
  and create a comprehensive CLAUDE.md file. Let me
  start by exploring the repository.

• Task(Analyze codebase structure)
  | Read(README.md)

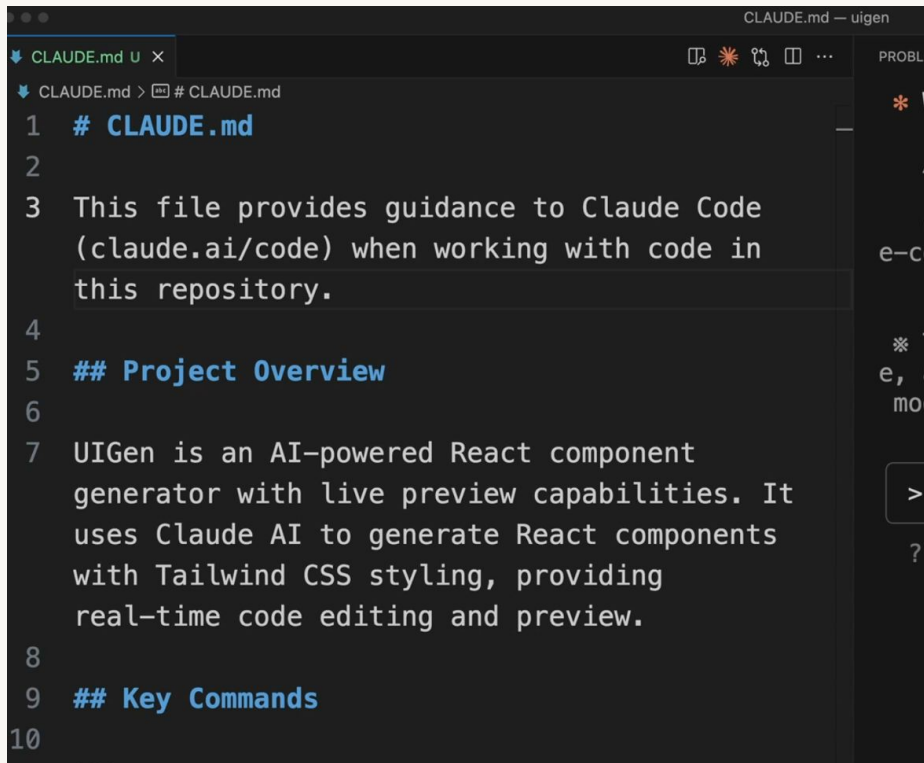
* Jiving... (14s · ✖ 262 tokens · esc to interrupt)

> █

? for shortcuts
```

- While writing files, Claude Code asks for permissions. Use **Shift + Tab** to enable auto accept

Claude.md



```
CLAUDE.md — uigen
# CLAUDE.md

This file provides guidance to Claude Code
(claude.ai/code) when working with code in
this repository.

## Project Overview

UIGen is an AI-powered React component
generator with live preview capabilities. It
uses Claude AI to generate React components
with Tailwind CSS styling, providing
real-time code editing and preview.

## Key Commands
```

The CLAUDE.md file

Guides Claude through your codebase, pointing out:

- Important commands
- Architecture
- Coding style

Allows you to give Claude specific or custom directions

- Helps Claude understand the codebase so that it can find relevant code more quickly
- Serves as a location where we can provide Claude some general guidance
- Update Claude md files by either modifying the file manually or use **#** for memory mode edit in Claude

Claude.md Files

Three Common Locations

CLAUDE.md

- Generated with `/init`
- Commit to source control
- Shared with other engineers

Project Level

CLAUDE.local.md

- Not shared with other engineers
- Contains personal instructions and customizations for Claude

Local Level

`~/.claude/CLAUDE.md`

- Used with all projects on your machine
- Contains instructions that you want Claude to follow on all projects

Machine Level

- Project Level: Keeps project specific directions that we want to hand off to Claude
- Local Level: Not committed to source control and keeps instructions Claude should follow for you
- Machine Level: Contains global instructions applied to all projects within the machine
- Add file mentions with `@` to add file content as context to Claude request. Can be used within md files

Solving Complex Tasks

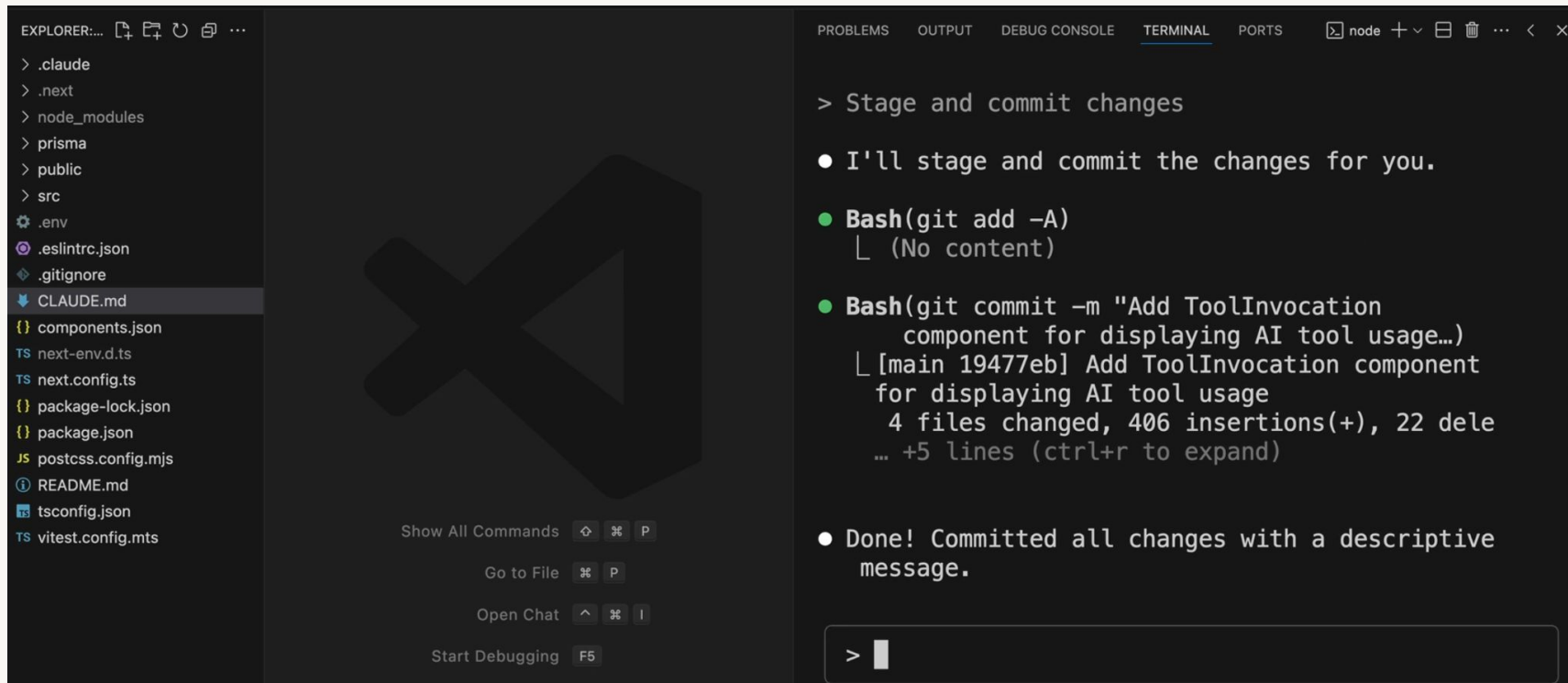
For solving complex multi-step tasks, there are two features to boost Claude Code's intelligence:

- **Planning mode:** Claude will do some thorough research on the contents of the project and come up with detailed plan to complete the task
 - After completing the plan, Claude shows the planned steps to complete the task which we can accept to implement or redirect it with what's missing
 - Use **Shift + Tab** twice to enable planning mode (or once if auto file edits is enabled)
- **Thinking modes:** Turns on Claude's extended thinking feature, allowing it to reason more about a particular task
 - Use these trigger phrases to enable (from least to most token budget to think with):
 - "Think"
 - "Think more"
 - "Think a lot"
 - "Think longer"
 - "Ultrathink"

Planning vs Thinking: When to use

- Planning mode is useful for:
 - tasks requiring a wide understanding of codebase and requires looking at different areas
 - Multi-step tasks
- Thinking mode is useful for:
 - Focussing on a tricky piece of logic
 - Troubleshooting a difficult bug
- Planning and Thinking modes can be used simultaneously
- Planning and Thinking mode consume additional tokens so there's a cost associated with using these features

Claude as Git Assistant



- Claude Code is a solid Git Assistant
- For example—ask it to stage and commit changes to generate a scripted commit message

Controlling Context

- Some methods to control context can be very effective in improving the response quality while changing between tasks or having long-running conversations with Claude
- **Interrupt:** Stop Claude with **Esc** to interrupt it and redirect it or give alternate instructions
- Interrupting with **Esc** and adding memory with **#** is a powerful way to prevent recurring mistakes from Claude in the future
- **Rewind conversation:** revert to a previous conversation state with **Esc + Esc** to prune the context not relevant to the current task—this helps Claude focus on a given task
- **Compact conversation:** **/compact <optional summarization instructions>** command takes all messages in the current conversation and summarizes them. Useful when Claude has learned a lot about the current task from conversation turns that can be used as context for the next task
- **Clear conversation:** **/clear** command dumps entire conversation history and allows to start from scratch. Useful for starting on a completely different task, unrelated to the current one

Controlling Context

Escape

- Interrupt Claude, allowing you to redirect or correct it.
- Also useful to fix issues with # memories.

Double-tap Escape

- Rewind the conversation to an earlier point in time.
- Allows you to maintain valuable context.

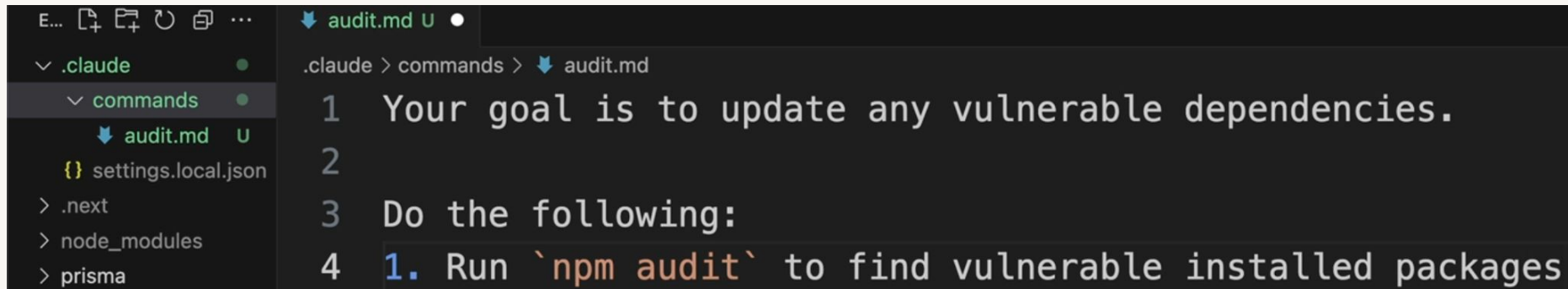
/compact

- Summarize the conversation and continue
- Helps Claude stay focused but remember what it has learned in the current session.

/clear

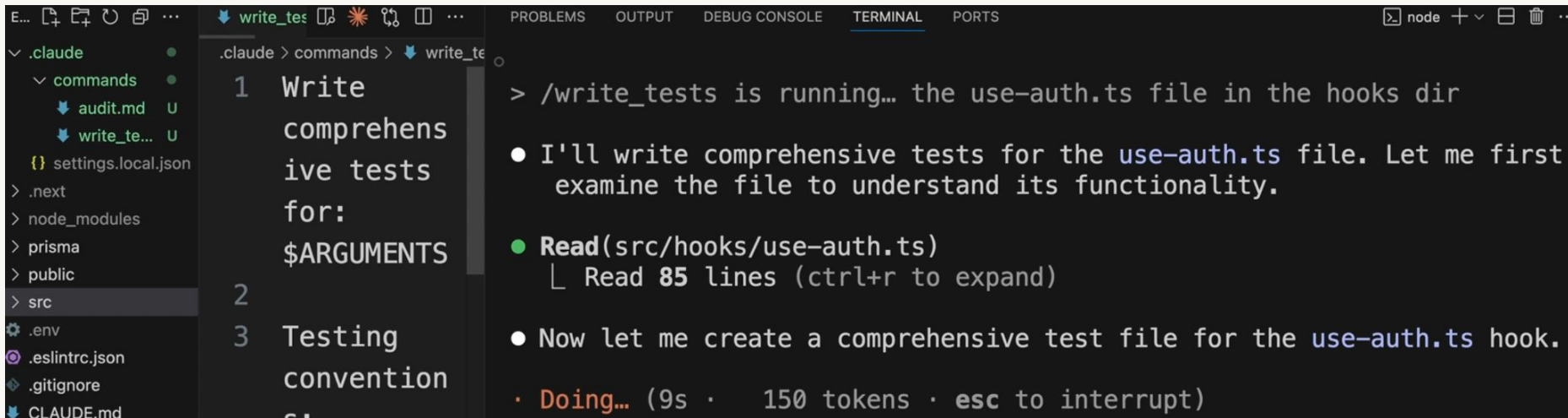
- Dumps current conversation history.
- Useful when switching between different tasks.

Custom Commands



- Use forward slash (/) to see all the built-in Claude Code commands
- In addition to default commands, we can also add in our own custom commands to automate repetitive tasks that we run frequently
- Add custom commands by adding commands directory into .claude and placing command files. Example—audit.md for audit custom command and then finally restart Claude Code

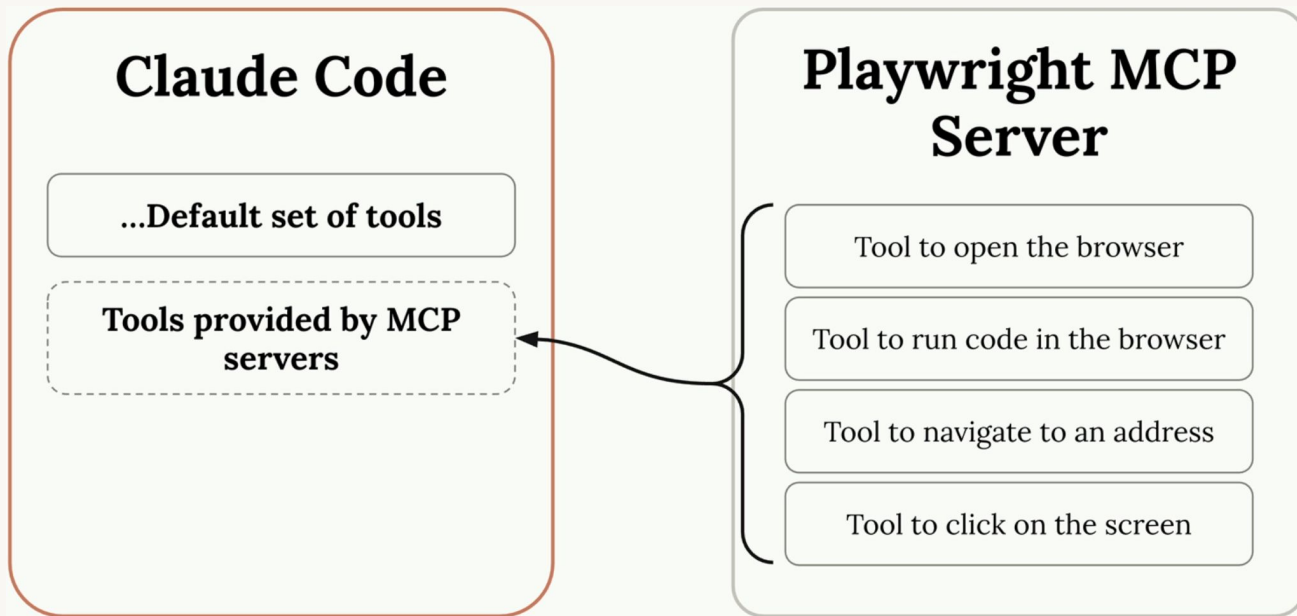
Custom Commands



```
.claude > commands > write_te
1 Write
comprehens
ive tests
for:
$ARGUMENTS
2
3 Testing
convention
> /write_tests is running... the use-auth.ts file in the hooks dir
• I'll write comprehensive tests for the use-auth.ts file. Let me first
  examine the file to understand its functionality.
• Read(src/hooks/use-auth.ts)
  | Read 85 lines (ctrl+r to expand)
• Now let me create a comprehensive test file for the use-auth.ts hook.
• Doing... (9s · 150 tokens · esc to interrupt)
```

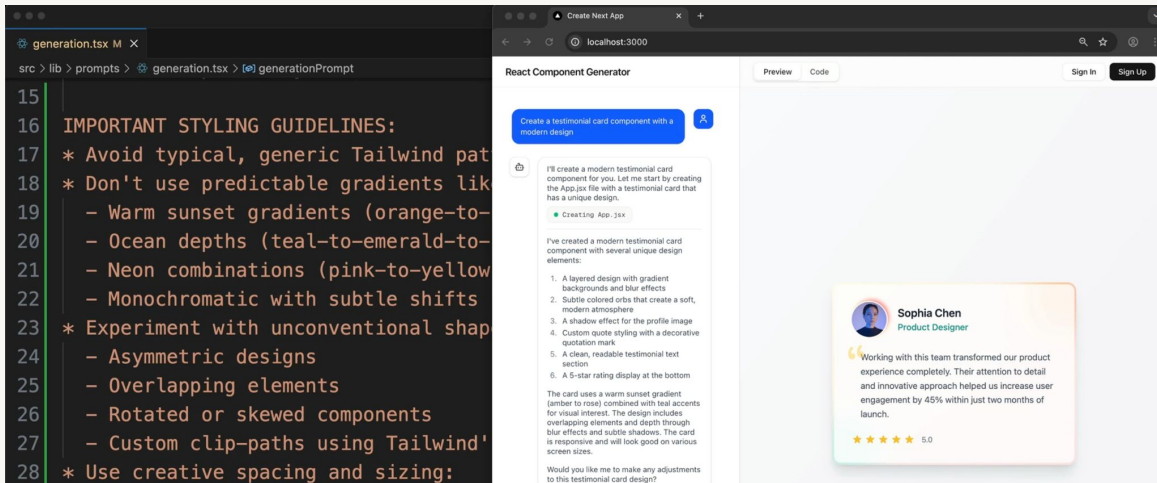
- Custom commands also supports arguments by adding a **\$ARGUMENTS** placeholder inside of command text md file
- While running the command, argument value will be passed to the placeholder dynamically.
- Example—writing tests for a particular file with filepath as argument. Note: while running command, input doesn't have to be a @filepath, but can be any string that we want to pass

Extending with MCP Servers



- Claude Code supports adding new tools and capabilities via MCP servers
- These MCP servers can be running locally on the machine or even remotely
- A popular MCP server, [Playwright](#) provides browser automation capabilities. It enables LLMs to interact with web pages through structured accessibility snapshots, bypassing the need for screenshots or visually-tuned models

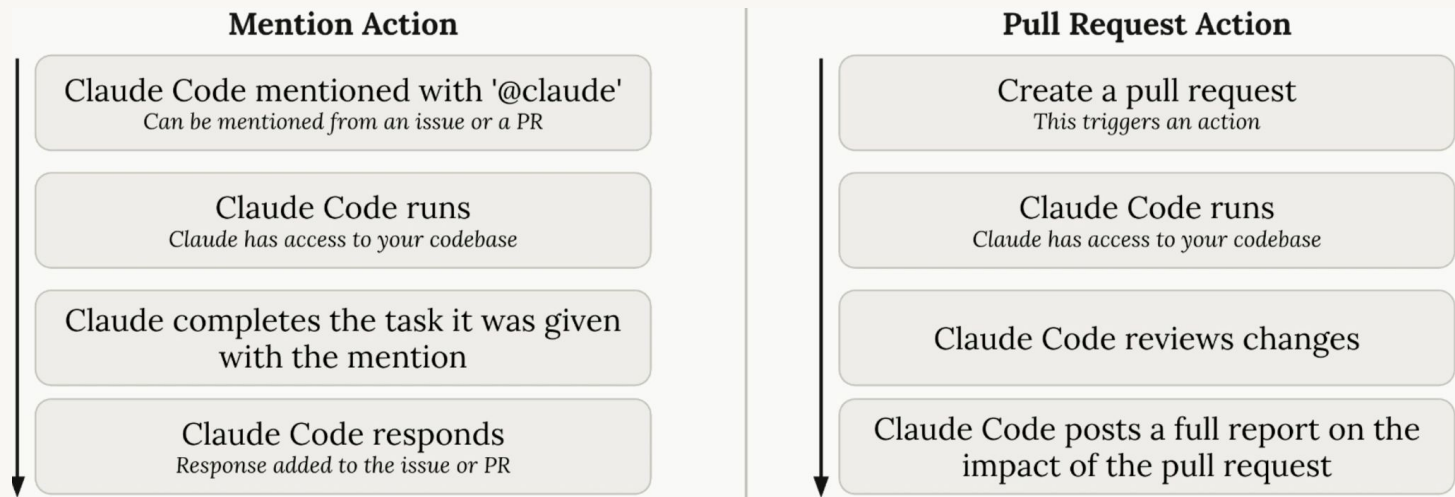
Browser Use with Playwright



- Run `claude mcp add playwright` to add Playwright MCP to claude
- Run `npx @playwright/mcp@latest` to start the MCP server locally on the machine
- Start Claude Code and ask it to open browser and navigate to the application on localhost:3000
- Before browser opens, Claude Code asks for permission for the tool to run
- For changing default permissions, update `.claude/settings.local.json` and add `"mcp__playwright"` inside `"allow"` array—this allows Claude Code to make use of the MCP server and all the tools in any way it wants without requiring permission every time

GitHub Integration

Default & BYO GitHub Actions



- Claude Code has official GitHub integration that allows Claude Code to run inside of a GitHub action
- Install by running **/install-github-app** and merge the auto-generated pull request which adds 2 github actions for Claude assistance. Both these actions can be customized:
 - Mention Action
 - Pull Request Action
- Adding additional / custom actions to trigger based on other types of events is also supported

Customizing @Mention

- Pull the merged two action config files on GitHub Repository to local with *git pull origin main*
- Two action config files will be pulled inside .github/workflows—one for mentions & another for PR
- To customize @claude to run the project and use Playwright MCP to access the app inside of the web browser, all inside GitHub Action:
 - Add npm run steps before Claude Code runs in the @claude mention action config file
 - Update Claude Code config to add custom instructions to add additional context that dev server is running and use Playwright MCP to access the app in the browser
 - Add configuration to setup Playwright MCP server
 - Since we are running Claude Code inside of an action, specifically list out all the permissions to grant Claude Code
 - If using MCP server, individually list out each tool from each MCP server that we want to allow
 - Test by opening an issue and tagging @claude

Customizing @Mention

```
! claude.yml M X
github > workflows > ! claude.yml
20 steps:
27 - name: Checkout repository
28   uses: actions/checkout@v4
29   with:
30     fetch-depth: 1
31
32 - name: Project Setup
33   run: |
34     npm run setup
35     npm run dev:daemon
36
37 - name: Run Claude Code
38   id: claude
39   uses: anthropics/claude-code-action@beta
40   with:
41     anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
42
43     custom_instructions: |
44       The project is already set up with all dependencies installed.
45       The server is already running at localhost:3000. Logs from it
46       are beign written to logs.txt. If needed, you can query the
47       db with the 'sqlite3' cli. If needed, use the mcp_playwright
48       set of tools to launch a browser and interact with the app.
```

```
48 set of tools to launch a browser and interact with the app.
49
50 mcp_config: |
51   {
52     "mcpServers": {
53       "playwright": {
54         "command": "npx",
55         "args": [
56           "@playwright/mcp@latest",
57           "--allowed-origins",
58           "localhost:3000;cdn.tailwindcss.com;esm.sh"
59         ]
60       }
61     }
62   }
63
64 allowed_tools: "Bash(npm:*),Bash(sqlite3:*),mcp_playwright_browser_
```

Customizing @Mention

Open Toggle Buttons #2

Here are the ones I'm talking about:

Preview Code

@claude can you take a look at this? Just test them out and verify that clicking them toggles between the code and preview tabs.

Create sub-issue

claude bot now · edited by claude

Testing Toggle Buttons

I'll test the toggle buttons and verify they work correctly for switching between code and preview tabs.

Todo List:

☒ Navigate to the application at localhost:3000

☒ Test toggle button functionality

☒ Identify any issues with the toggle behavior

☐ Examine code if problems are found

☐ Fix any issues discovered

☐ Commit changes if needed

Test Results

☒ Toggle Buttons Work Correctly

• Performed extensive testing with both empty state and generated content

No labels

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Code with Copilot Agent Mode

Create a branch for this issue or link a pull request.

Notifications

Customize

Unsubscribe

You're receiving notifications because you're subscribed to this thread.

Participants

Transfer issue

Lock conversation

Pin issue

Hooks

Hooks

- Allows to run commands before or after Claude attempts to run a tool or optionally, block Claude's action
- A few ways hooks can be used to implement these functionalities:

Run a code formatter after
Claude edits a file

Stop Claude from editing or
reading a particular file

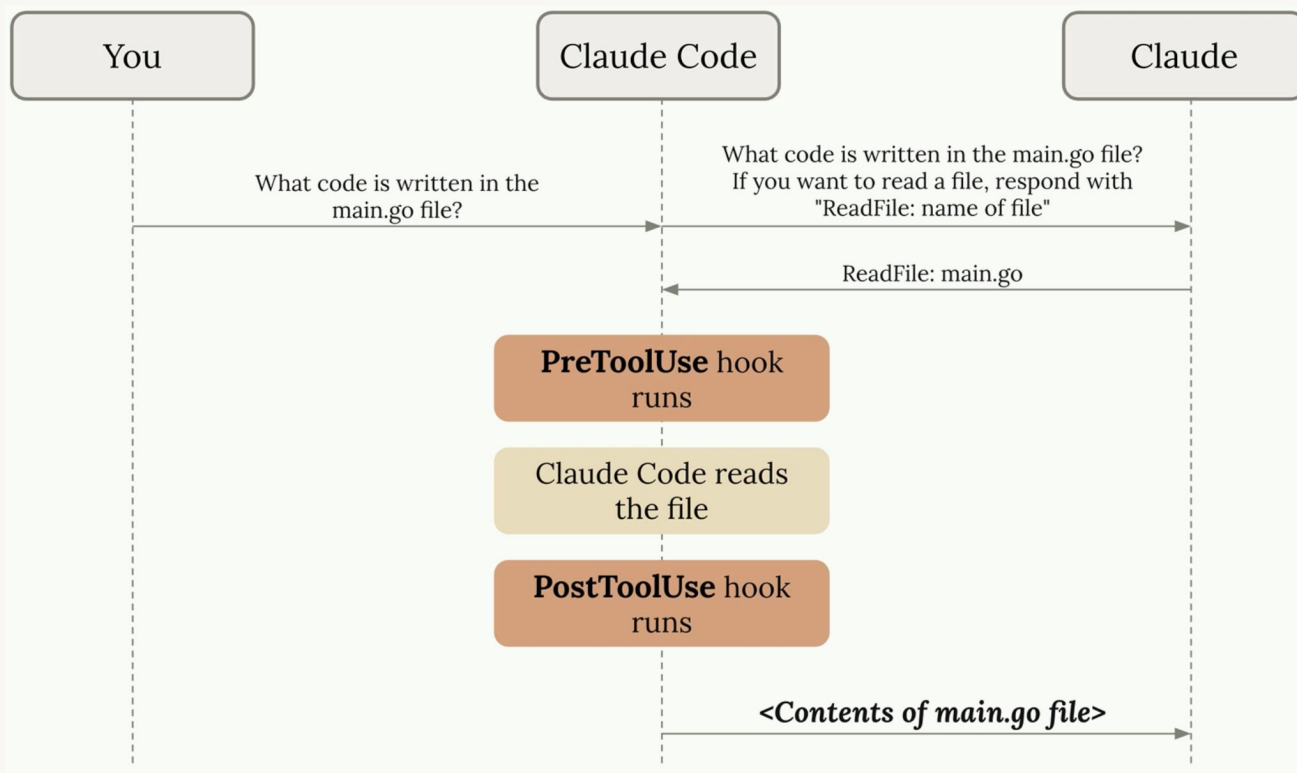
Check for TODO comments
in code that Claude writes
and add them to a log file

Run tests automatically
after a file is changed

Block file edits that add
variables that don't follow
naming conventions

Block deprecated function
usage

How Hooks Work?



- Hooks give Claude Code the ability to execute code just before or after the tool execution
- Pre-tool use hooks are the hooks that run before a tool and Post-tool use hooks run after a tool

Hook Definitions

Hook Definitions

Defined in...

Global	~/.claude/settings.json
Project	.claude/settings.json
Project (not committed)	.claude/settings.local.json

Write **by hand** or by using the **/hooks** command

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Read",
        "hooks": [
          {
            "type": "command",
            "command": "node /home/hooks/read_hook.ts"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Write|Edit|MultiEdit",
        "hooks": [
          {
            "type": "command",
            "command": "node /home/hooks/edit_hook.ts"
          }
        ]
      }
    ]
  }
}
```

Whenever Claude attempts to read a file ...

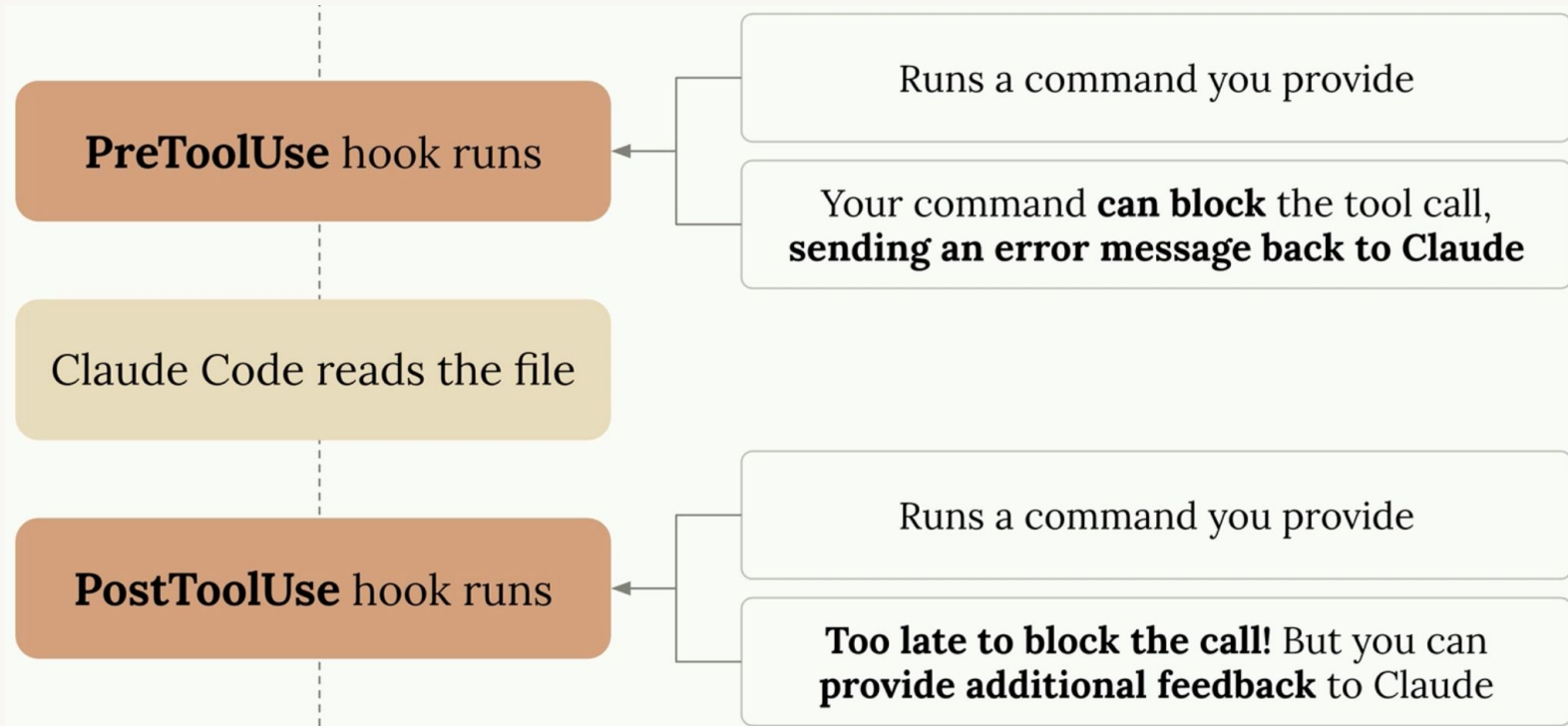
... run this command

After write, edit, or multiedit tools are executed ...

... run this command

- **Matcher:** indicates which tool use types we are looking for. Example—uses of read tool
- **Command:** commands to run via hooks before or after tool execution

What Hooks are Intended to Do?



- Pre-tool hook: Intercepts Claude's intended tool action before execution. Lets you inspect, modify, or block the tool call and return an error or guidance to Claude
- Post-tool hook: Runs after the tool executes. Enables follow-up actions such as formatting the edited file, running code quality checks, and feeding structured feedback back to Claude to update the file that it just wrote to

Hooks Security Best Practices

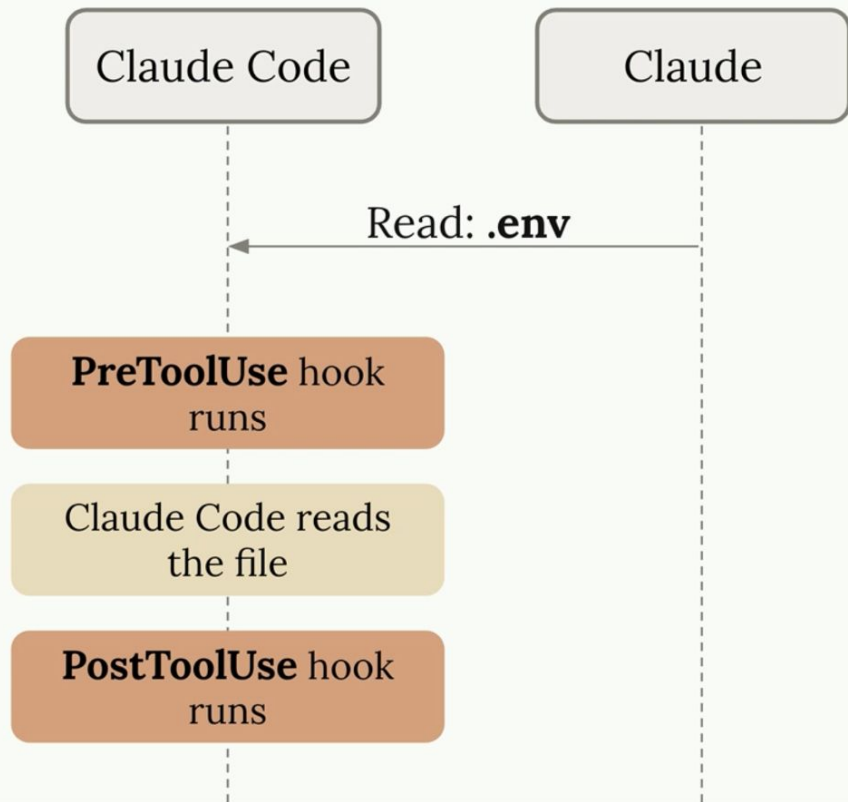
- **Validate and Sanitize Inputs:** Don't trust input data blindly
- **Always quote shell variables:** Use "\$VAR" instead of \$VAR
- **Block path traversal:** Check for .. in file paths
- **Use Absolute Paths:** Specify full paths for scripts rather than relative paths
 - This helps mitigate path interception and binary planting attacks
 - However, Absolute paths differ across machines making sharing settings.json files difficult
 - Solution is to use Template-based config and automated setup:
 - settings.example.json uses a \$PWD placeholder instead of hardcoded absolute paths
 - running npm run setup installs dependencies and executes init-claude.js, which replaces \$PWD with the local absolute path and generates settings.local.json
- **Skip sensitive files:** Avoid .env, .git/, keys, etc.

Defining Hooks

Stop .env file reads

Building a Hook

- 1 **Decide on a PreToolUse or PostToolUse hook**
- 2 Determine which type of tool calls you want to watch for
- 3 Write a command that will receive the tool call
- 4 If needed, command should provide feedback to Claude



Defining Hooks

Building a Hook

1

Decide on a PreToolUse or PostToolUse hook

2

Determine which type of tool calls you want to watch for

3

Write a command that will receive the tool call

4

If needed, command should provide feedback to Claude

Tool Names

Tool Name	Purpose
Read	Read a file
Edit, MultiEdit	Edit an existing file
Write	Create a file and write to it
Bash	Execute a command
Glob	Find files/folders based upon a pattern
Grep	Search for content
Task	Create a sub-agent to complete a particular task
WebFetch, WebSearch	Search or fetch a particular page

- Since we can add custom tools via various MCP servers in addition to built-in ones, you can ask Claude Code to **list out names of all the tools you have access to in bullet points**
- For blocking .env reads, we want to watch **Read** and **Grep** as these tools can read file contents

Defining Hooks

Building a Hook

- 1 Decide on a PreToolUse or PostToolUse hook
- 2 Determine which type of tool calls you want to watch for
- 3 **Write a command that will receive the tool call**
- 4 If needed, command should provide feedback to Claude

Tool Call Data

```
{
  "session_id": "2d6a1e4d-6...",
  "transcript_path": "/Users/sg/...",
  "hook_event_name": "PreToolUse",
  "tool_name": "Read",
  "tool_input": {
    "file_path": "/code/queries/.env"
  }
}
```

Standard In

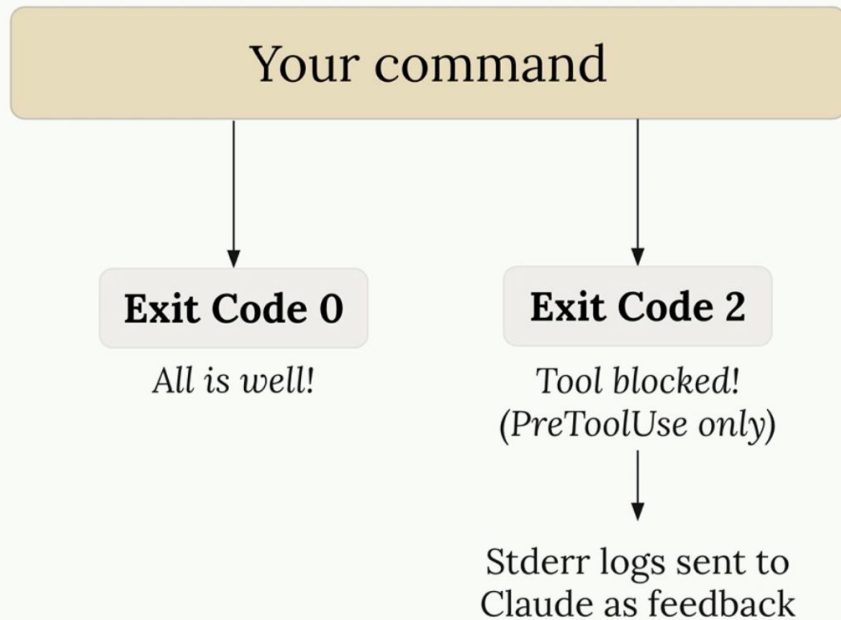
Your command

- Write out a command, Claude automatically executes it and on stdin to that process, Claude feeds it JSON tool call
- Command receives this via stdin, parses the JSON to fetch tool name, tool inputs, and decide what we want to do with this tool call

Defining Hooks

Building a Hook

- 1 Decide on a PreToolUse or PostToolUse hook
- 2 Determine which type of tool calls you want to watch for
- 3 Write a command that will receive the tool call
- 4 **If needed, command should provide feedback to Claude**



- Exit code provides a signal back to Claude Code:
 - Exit Code=0: allow tool call to occur
 - Exit Code=2: block tool call. stderr logs generated inside the command is sent as feedback to Claude—this helps deny the tool call and give Claude reason at the same time

Implementing Hooks

```
JS read_hook.js •
hooks > JS read_hook.js > main
1 async function main() {
2   const chunks = [];
3   for await (const chunk of process.stdin) {
4     chunks.push(chunk);
5   }
6   const toolArgs = JSON.parse(Buffer.concat(chunks).toString());
7
8   // readPath is the path to the file that Claude is trying to read
9   const readPath =
10     toolArgs.tool_input?.file_path || toolArgs.tool_input?.path || "";
11
12   // TODO: ensure Claude isn't trying to read the .env file
13   if(readPath.includes('.env')) {
14     console.error("You cannot read the .env file")
15     process.exit(2);
16   }
17 }
18
19 main();
```

Read from stdin and parse data as JSON. toolArgs holds Tool call data

Exit with code 2 and provide stderr feedback to claude on the failure reason

Implementing Hooks

```
1 {
2   "hooks": {
3     "PreToolUse": [
4       {
5         "matcher": "Read|Grep",
6         "hooks": [
7           {
8             "type": "command",
9             "command": "node ./hooks/read_hook.js"
10          }
11        ]
12      },
13      {
14        "matcher": "Write|Edit|MultiEdit",
15        "hooks": [
16          {
17            "type": "command",
18            "command": "node /Users/sgrider/code/enabements/claude-code-course/pr",
19            "timeout": 300
20          }
21        ]
22      }
23    ]
24  }
```

Whenever Claude attempts to read or grep files

... run this node js command

- Restart Claude Code after defining hooks and command/script for changes to take effect

Implementing Hooks

> Read the .env file

- I'll read the .env file for you.

- **Read(.env)**

- └ Error: Read operation blocked by hook:
- [node ./hooks/read_hook.js]: You cannot
read the .env file

> Try the grep tool to read it

- **Search(pattern: ".*", path: ".env",
output_mode: "content")**

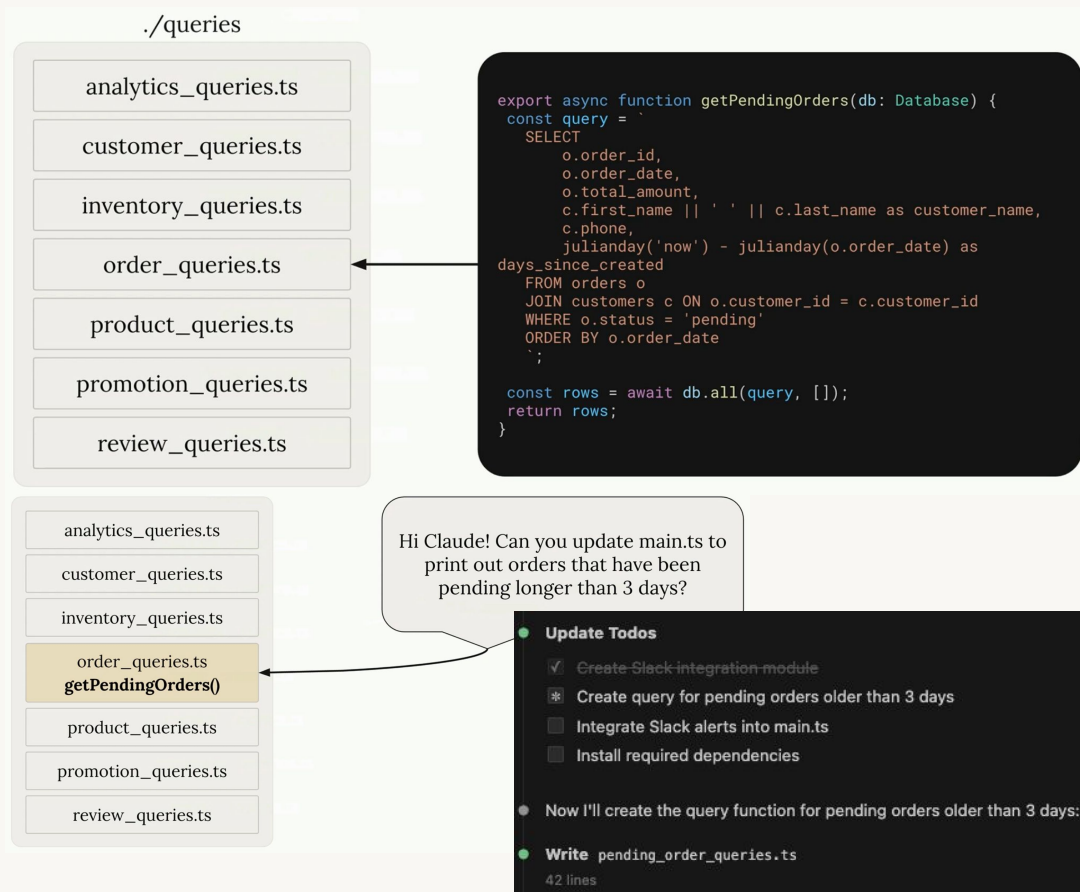
- └ Error: Grep operation blocked by hook:
- [node ./hooks/read_hook.js]: You cannot
read the .env file

- The .env file is also protected from the Grep tool by the same read hook.

Useful Hooks!

- There are some weaknesses in Claude Code that we can overcome using hooks:
- File Edit did not capture dependencies between files and introduced compilation errors:
 - It can apply local function changes, but does not update usages across the codebase
 - Mismatched signatures lead to TypeScript compile errors, requiring hooks for safe refactors
 - This can be fixed by a PostToolUse hook where we run typescript type checker everytime we edit a TypeScript file and see if there are any errors
 - We can feed output of tsc check to Claude Code as a signal to indicate type error, if any
 - Similar setup works for any other typed language. For untyped language, a similar setup can be done leveraging tests instead of a type checker

Useful Hooks!



- In an ideal world, if Claude is asked to update main.ts and print out pending orders with > 3 days:
 - Find `order_queries.ts`
 - Find existing `getPendingOrders()` function
 - Reuse `getPendingOrders()` and filter on the result set having `days_since_created` column
 - Not create a new query using SQL from `getPendingOrders()` with additional 3 days filter
- In larger projects; if finding pending order is only a sub-task, Claude doesn't stay focussed and tries to create a new query file

Useful Hooks!

Claude uses **Write**, **Edit**, or **MultiEdit** to modify a file in the `./queries` dir



PreToolUse Hook

Programmatically launch a *separate copy* of Claude Code

Ask it to research the queries dir and see if a similar query already exists

If so, provide this feedback to Claude, giving it the opportunity to fix the situation

- Whenever Claude attempts to use Write/Edit/MultiEdit tool to modify files within queries directory, run a pre-tool hook
- Within the hook; we do the following:
 - Programmatically run a separate instance of Claude Code
 - Take a look at the intended change and look at some of the existing code inside the queries directory
 - Check if a similar query is already inside there
 - If there's an existing similar query, take that Feedback and send to the original instance of Claude Code
 - Based on the feedback, Claude Code tries with an updated strategy and tool call to reuse existing query

Useful Hooks!

```
6 const REVIEW_DIR = "src/queries";
7
8 async function main() {
9   // Read JSON input from stdin
10   const input = await new Promise((resolve) => {
11     let data = "";
12     process.stdin.on("data", (chunk) => (data += chunk));
13     process.stdin.on("end", () => resolve(data));
14   });
15
16   const hookData = JSON.parse(input);
17   const toolInput = hookData.tool_input;
18
19   // Check if this is a file modification in ./queries
20   const filePath = toolInput.file_path || toolInput.path;
21   if (!filePath) {
22     process.exit(0);
23   }
24
25   // Normalize paths for comparison
26   const normalizedFilePath = path.resolve(filePath);
27   const queriesDir = path.resolve(process.cwd(), REVIEW_DIR);
28
29   // Check if file is within queries directory (handles subdirectories too)
30   if (!normalizedFilePath.startsWith(queriesDir + path.sep)) {
31     process.exit(0);
32   }
33
34   // Prepare prompt for analysis
35   const newContent =
36     toolInput.content || toolInput.contents || toolInput.new_string;
37   const prompt = `You are reviewing a proposed change to a database query file.
38 Your task is to analyze if the new or modified query functions could be
39 accomplished by reusing or slightly modifying existing query functions.
40
41 Within reason, we want to prevent duplicate queries from being added into this project,
42 so you are seeing if the proposed change will duplicate any existing functionality.
43
44 File: ${filePath}
45 New content:|
46 <new_content>
47 ${newContent}
48 </new_content>
```

Parse Tool Call Data

Check if the change
was made to the
queries dir

Please research and analyze the existing queries in the ./queries directory and:

1. Identify any new query functions being added in this change
2. For each new query function, determine if it could be accomplished by:
 - Using an existing query function as-is
 - Slightly modifying an existing query function, perhaps by adding additional arguments or expanding a select statement

If yes, provide specific feedback on which existing functions could be used instead. Be concise and specific. If no, just say "Changes look appropriate.";

```
const messages = [];
for await (const message of query({
  prompt,
})) {
  messages.push(message);
}
```

Make a call to Claude Code using Claude
Code Typescript SDK

```
// Extract the analysis result
const resultMessage = messages.find((m) => m.type === "result");
if (!resultMessage || resultMessage.subtype !== "success") {
  process.exit(0);
}
```

```
// If changes are appropriate, allow them
if (resultMessage.result.includes("Changes look appropriate")) {
  process.exit(0);
}
```

```
// Otherwise, block with feedback
console.error(`Query duplication detected:\n\n${resultMessage.result}`);
process.exit(2);
```

Check Claude Response if
change is duplicate or not.
If duplicate, exit with error
code 2 to tell Claude Code
to make changes

```
main().catch((err) => {
  console.error(`Hook error: ${err.message}`);
  process.exit(1);
});
```

Prompt asking Claude to review the
proposed change

Prompt asking Claude to review the
proposed change

Useful Hooks!

- One downside of these hooks can be additional time and expense every time an edit is being made in the queries directory since it has to make another Claude call with some prompt
- However, the upside is we end up with a lot less duplicate code
- You can limit the scope of the directories to ensure hooks are making Claude call only when edit is being made to the queries directory

Another Useful Hook!

- Tools beyond PreToolUse and PostToolUse:
 - **Notification:** runs when Claude Code sends a notification, which occurs when Claude needs permission to use a tool, or after Claude Code has been idle for 60 seconds
 - **Stop:** runs when Claude Code has finished responding
 - **SubagentStop:** runs when a subagent has finished. Subagents are displayed as a “Task” in the UI
 - **PreCompact:** runs before a compact operation occurs, either manual or automatic
 - **UserPromptSubmit:** runs when user submits a prompt, before Claude processes it
 - **SessionStart:** runs when starting or resuming a session
 - **SessionEnd:** runs when a session ends
- stdin input to commands will change based on the type of hook being executed (PreToolUse, PostToolUse, Notification, etc.)
- tool_input contained in that will differ based on the tool that was called (in case of PreToolUse and PostToolUse hooks)

Examples stdin Inputs for tools

PreToolUse hook: ToDoWrite tool

```
{
  "session id":
  "9ecf22fa-edf8-4332-ae85-b6d5456eda64",
  "transcript path": "<path to transcript>",
  "hook_event_name": "PostToolUse",
  "tool_name": "ToDoWrite",
  "tool input": {
    "todos": [{ "content": "write a readme",
"status": "pending", "priority": "medium",
"id": "1" }]
  },
  "tool response": {
    "oldTodos": [],
    "newTodos": [{ "content": "write a
readme", "status": "pending", "priority":
"medium", "id": "1" }]
  }
}
```

Stop hook

```
{
  "session id":
  "af9f50b6-f042-4773-b3e2-c3a4814765ce",
  "transcript path": "<path to transcript>",
  "hook_event_name": "Stop",
  "stop_hook_active": false
}
```

Hook to log stdin

```
"PostToolUse": [ // Or "PreToolUse" or "Stop", etc
{
  "matcher": "*",
  "hooks": [
    {
      "type": "command",
      "command": "jq . > post-log.json"
    }
  ]
},
]
```

- As the stdin input to the command differs significantly based on the hook, it can make writing hooks challenging
- This hook will write input to this hook to a log file
- This allows users to inspect the stdin for that hook and easier to implement the command

Claude Code SDK

Claude Code SDK

Claude Code SDK

- Available for Typescript, Python, and via the CLI
- Runs Claude Code programmatically
- Same Claude Code!
- Inherits all the settings of an instance of Claude Code launched in the same directory
- Has read-only permissions by default
- Useful as part of a larger pipeline or tool

ANTHROPIC

CLI

```
claude -p "Look for duplicate queries"
```

Typescript

```
import { query, SDKMessage } from "@anthropic-ai/claude-code";  
  
const prompt = "Look for duplicate queries";  
  
for await (const message of query({prompt})) {  
  console.log(message);  
}
```

Python

```
import anyio  
from claude_code_sdk import query  
  
async def main():  
    prompt = "Look for duplicate queries"  
    async for message in query(prompt=prompt):  
        print(message)  
  
anyio.run(main)
```

Claude Code SDK

- Claude Code SDK is most useful as part of larger pipeline or tool or inside a hook to add intelligence to a given task inside that workflow
- By default, SDK only has read abilities—read files, do grep operations. Can't create and edit files
- To give it write permissions, configure write permissions while making the Claude SDK call or add permissions within settings.json file inside .claude directory

```

s schema.ts    TS main.ts    JS query_hook.js    TS sdk.ts    •    {} package.json > ...
TS sdk.ts > ...
1  // Note: the "@anthropic-ai/claude-code" package has been renamed
2  // to "@anthropic-ai/claude-agent-sdk"
3  import { query } from "@anthropic-ai/claude-agent-sdk";
4
5  // const prompt = "Look for duplicate queries inside the ./src/queries directory"
6  const prompt = "Add description to the package.json file in the project root."
7
8  for await (const message of query({
9    prompt,
10   options:{
11     allowedTools: ["Edit"]
12   }) {
13   }
14   console.log(JSON.stringify(message, null, 2));
15 }
16
1 {} package.json > ...
1  {
2    "name": "queries",
3    "version": "1.0.0",
4    "description": "A TypeScript project for database queries using Claude Agent SDK and SQLite",
5    "main": "index.js",
6    >Debug
7    "scripts": {
8      "setup": "npm install && node ./scripts/init-claude.js",
9      "test": "echo \"Error: no test specified\" && exit 1",
10     "sdk": "tsx sdk.ts"
11   },
12   "keywords": [],
13   "author": "",
14   "license": "ISC",
15   "type": "module",
16   "dependencies": {
17     "@anthropic-ai/claude-agent-sdk": "^0.1.5",
18     "gslack/webhook": "^7.0.6",
19     "@types/node": "^24.0.13",
20     "sqlite": "^5.1.1",
21     "sqlite3": "^5.1.7",
22     "tsx": "^4.20.3",
23     "typescript": "^5.8.3"
24   }
25 }
```

Conclusion

Wrap up and next steps

Watch for changes and updates

Keep an eye on anthropic.com/claude-code

Experiment!

Customize with commands, CLAUDE.md, and MCP servers

Automate!

Use the Github integration to delegate common tasks to Claude